

Oportunidades de Desenvolvimento —

Módulo de Email

Listei abaixo as oportunidades organizadas por categoria, com contexto técnico e arquivos envolvidos. Priorizei do maior para o menor impacto percebido.

1. Novas Funcionalidades

1.1. Templates customizáveis por DAG

Oportunidade: Permitir que cada YAML aponte para um template HTML próprio (ex: `template: "juridico.html"`).

Arquivos: `src/schemas.py` (campo em `ReportConfig`), `src/notification/email_sender.py` , novo diretório `src/notification/templates/custom/` .

Cuidados: Sandbox Jinja2, whitelist de templates, validação no `dag_generator` .

1.2. Suporte a múltiplos destinatários com regras condicionais

Oportunidade: Enviar relatórios diferentes para destinatários diferentes (ex: jurídico recebe só `pubtype: Portaria` , TI recebe só `term: tecnologia`).

Arquivos: `src/schemas.py` , `src/dou_dag_generator.py` , `src/notification/email_sender.py` .

Impacto: Reduz a proliferação de DAGs duplicadas.

1.3. Link direto para a DAG no Airflow no rodapé do email

Oportunidade: Incluir no rodapé um botão “Ver execução no Airflow” com URL dinâmica (`AIRFLOW__API__BASE_URL` + `dag_id` + `run_id`).

Arquivos: `src/notification/email_sender.py` (precisa receber `dag_run` via XCom), `src/notification/templates/dou_template.html` .

1.4. Versão “digest semanal/mensal” consolidada

Oportunidade: Quando `date: SEMANA` ou `date: MES`, agrupar resultados por dia no corpo do email (hoje vem tudo misturado).

Arquivos: `src/notification/email_sender.py` (`_generate_email_content`),
`src/notification/templates/dou_template.html` .

1.5. Preview do email antes do envio (dry-run)

Oportunidade: Task opcional que gera o HTML e o salva como attachment no XCom ou envia só para um email de teste, sem disparar para a lista final.

Arquivos: `src/dou_dag_generator.py` (nova task condicional),
`src/notification/email_sender.py` .

1.6. Suporte a anexos além de CSV (ex: JSON, PDF do relatório)

Oportunidade: Permitir anexar o relatório renderizado em PDF ou os dados em JSON.

Arquivos: `src/schemas.py` (`attach_pdf` , `attach_json`),
`src/notification/email_sender.py` .

2. Melhorias Visuais e de UX

2.1. Modo escuro (dark mode) no template

Oportunidade: Adicionar `@media (prefers-color-scheme: dark)` no `dou_template.html` .

Arquivo: `src/notification/templates/dou_template.html` .

Atenção: Outlook não suporta dark mode — precisa de fallback.

2.2. Botão “Copiar termo” ou “Filtrar por termo” no email

Oportunidade: Cada bloco de termo poderia ter um âncora interna para navegação rápida em relatórios grandes.

Arquivo: `src/notification/templates/dou_template.html` (gerar `id` único por bloco + índice no topo).

2.3. Contador de resultados no cabeçalho de cada busca

Oportunidade: Exibir “Encontradas 23 publicações para ‘dados abertos’” no header de cada bloco.

Arquivos: `src/notification/email_sender.py` ,
`src/notification/templates/dou_template.html` .

2.4. Melhorar rendering em Outlook/Gmail

Oportunidade: O template atual usa Flexbox em alguns lugares (`.group-link` , `.footer`), que o Outlook não suporta. Substituir por tables aninhadas.

Arquivo: `src/notification/templates/dou_template.html` .

Validação: Testar com Litmus ou similar.

2.5. Destaque visual para publicações com score muito alto

Oportunidade: Hoje há tags de relevância, mas nenhuma distinção no layout.

Publicações com score > 20 poderiam ter borda lateral colorida.

Arquivo: `src/notification/templates/dou_template.html` .

3. Integrações e Extensibilidade

3.1. Webhook de pós-envio (callback)

Oportunidade: Após enviar o email, disparar um webhook (ex: para um sistema de ticket ou dashboard) com metadados do relatório.

Arquivos: `src/schemas.py` (`post_send_webhook`), `src/notification/email_sender.py` .

3.2. Integração com Microsoft Teams nativa

Oportunidade: Hoje Teams só via Apprise. Um `TeamsSender` dedicado permitiria Adaptive Cards.

Arquivo: novo `src/notification/teams_sender.py` , seguir padrão de `SlackSender` .

3.3. Integração com o novo DAG Generator web

Oportunidade: O `dag_generator/app.py` hoje não expõe o campo `header_text` , `footer_text` , `ai_report_config` nem `notification` . Ampliar o schema Pydantic do gerador.

Arquivo: `dag_generator/app.py` (`ReportConfig` , `SearchConfig`),
`dag_generator/static/gerador_yaml.html` .

3.4. Exportar relatório para Google Sheets / Notion

Oportunidade: Além de CSV anexo, enviar os dados diretamente para uma planilha ou banco de conhecimento.

Arquivos: novo `src/notification/sheets_sender.py` ou extensão do `NotificationSender` .

4. Performance e Escalabilidade

4.1. Geração de CSV em streaming

Oportunidade: Hoje o CSV é todo montado em memória via `NamedTemporaryFile` . Para relatórios muito grandes (>10k publicações), usar escrita em streaming.

Arquivo: `src/notification/email_sender.py` (`get_csv_tempfile` , `convert_report_to_dataframe`).

4.2. Cache de template Jinja2

Oportunidade: O `TemplateManager` recia o `Environment` a cada email. Em DAGs com múltiplos destinatários ou relatórios, cache pode ajudar.

Arquivo: `src/notification/templateManager.py` .

4.3. Limite de tamanho do HTML

Oportunidade: Gmail corta emails acima de 102KB. Adicionar aviso e opção de truncar ou dividir em múltiplos emails quando o HTML passar desse limite.

Arquivo: `src/notification/email_sender.py` .

5. Segurança

5.1. Sanitizar `header_text` e `footer_text`

Oportunidade: Hoje esses campos são renderizados com `| safe` no template — um YAML malicioso pode injetar JS. Aplicar `nh3.clean()` como no filtro `markdown` .

Arquivos: `src/notification/templateManager.py` (novo filtro `safe_html`), `src/notification/templates/dou_template.html` .

5.2. Validação de domínios nos emails destinatários

Oportunidade: `EmailStr` do Pydantic já valida formato, mas não domínio. Adicionar opção de whitelist de domínios (`allowed_domains:["@gestao.gov.br"]`) para evitar

vazamentos.

Arquivo: `src/schemas.py` (`ReportConfig`).

5.3. Ofuscação de emails no HTML

Oportunidade: Para relatórios que podem ser compartilhados, ofuscar emails no HTML (ex: `destino [at] dominio.gov.br`) para evitar scraping.

Arquivo: `src/notification/templates/dou_template.html` .

6. Testabilidade e Qualidade

6.1. Testes de rendering cross-client

Oportunidade: Suite de testes que renderiza o template e valida a estrutura HTML com BeautifulSoup, garantindo que elementos críticos (abstract, data, link) estão presentes.

Arquivo: novo `tests/email_template_rendering_test.py` .

6.2. Snapshot tests do HTML gerado

Oportunidade: Congelar o HTML esperado para configurações típicas (com/sem executive summary, com/sem filtros, com/sem relevância) e detectar regressões visuais.

Arquivo: novo `tests/email_snapshot_test.py` + fixtures em `tests/fixtures/` .

6.3. Testes para caminhos de falha do SMTP

Oportunidade: Hoje não há teste para `send_email` levantando exceção (SMTP fora do ar, destinatário inválido).

Arquivo: `tests/email_sender_test.py` .

6.4. Cobrir casos-limite do CSV

Oportunidade: Testar CSV com múltiplos grupos, departamentos, termos com caracteres especiais (vírgula, aspas), publicações sem abstract.

Arquivo: `tests/email_sender_test.py` .

7. Observabilidade

7.1. Métricas de envio

Oportunidade: Exportar métricas via XCom ou log estruturado: número de emails enviados, tamanho do HTML, tempo de renderização, quantidade de resultados.

Arquivo: `src/notification/email_sender.py` .

7.2. Log do HTML renderizado em caso de falha

Oportunidade: Quando `send_email` falha, salvar o HTML em `mnt/` para debug.

Arquivo: `src/notification/email_sender.py` .

7.3. Header customizado no email para rastreamento

Oportunidade: Adicionar `X-RoDOU-DAG-ID` e `X-RoDOU-Run-ID` como headers do email, permitindo filtrar no servidor SMTP.

Arquivo: `src/notification/email_sender.py` (passar `headers=` para `send_email`).

Sugestão de Roadmap

Prioridade	Oportunidades
Alta (quick wins com impacto)	3.3 (DAG Generator), 5.1 (sanitização), 2.4 (Outlook), 6.1-6.3 (testes)
Média (valor agregado)	1.3 (link Airflow), 1.4 (digest por dia), 2.3 (contador), 4.3 (limite Gmail), 7.1-7.3 (observabilidade)
Média-baixa (evolução)	1.1 (templates custom), 1.2 (destinatários condicionais), 3.2 (Teams), 2.1 (dark mode)
Backlog	1.5 (preview), 1.6 (PDF/JSON), 3.4 (Sheets/Notion), 4.1-4.2 (performance)
